

TypeForge

Converts Markdown files with LaTeX math into production-ready PDFs using Pandoc, MathJax, and Puppeteer.

What It Does

This tool transforms `.md` files into A4 PDFs with: - **LaTeX math** rendered as selectable, scalable text (not images) - **Code syntax highlighting** - **Embedded images** with width control - **Custom typography** via Google Fonts (Barlow, JetBrains Mono)

Pipeline

Each Markdown file passes through three stages:

1. **Pandoc** converts Markdown to standalone HTML
2. **MathJax** pre-renders LaTeX math to static CHTML
3. **Puppeteer** renders the HTML to PDF via headless Chromium

```
src/md/*.md → temp/*_step1.html → temp/*_step2.html → output/*.pdf
```

Prerequisites

- [Node.js](#) ≥ 18
- [Pandoc](#) (must be in PATH)
- PowerShell (`pwsh` or `powershell`)

Installation

```
npm install
```

Usage

Place `.md` files in `src/md/`, then run:

```
.\md2pdf.ps1
```

Options

Parameter	Description
<code>-MarkdownFolder <path></code>	Override input directory
<code>-OutputFolder <path></code>	Override output directory
<code>-KeepTemp</code>	Preserve intermediate HTML files for debugging

Project Structure

```
├─ md2pdf.ps1           # Main entry point
├─ defaults.yaml        # Pandoc configuration
├─ scripts/
│   ├─ prerender-math.js # MathJax v4 rendering via @mathjax/src
│   ├─ render-pdf.js     # Puppeteer PDF generation
│   └─ test-runner.js    # Node test suite
├─ src/
│   ├─ md/               # Input Markdown files
│   └─ css/mdcss.css     # Optional custom stylesheet
```

```
| └─ img/           # Image assets
| └─ output/        # Generated PDFs (gitignored)
| └─ temp/          # Intermediate HTML files (gitignored)
```

Supported Math Syntax

Type	Syntax
Inline	<code>\$x^2\$</code> or <code>\(x^2\)</code>
Display	<code>\$\$x^2\$\$</code> or <code>\[x^2\]</code>

`\tag{}` is automatically converted to `\text{(...)}` inside math spans to avoid MathJax rendering errors. Code blocks are left untouched.

Including Images

Use standard Markdown image syntax:

```
![Caption](src/img/photo.jpg)
```

For width control, use raw HTML attributes:

```

```

Note: Images must be in the `src/` directory or a subdirectory. Pandoc resolves paths relative to `resource-path` configured in `defaults.yaml`.

Example Image



A gray cat with yellow eyes looking to the left

PDF Format

- **Size:** A4
- **Margins:** 25mm on all sides

- **Body font:** Barlow
- **Code font:** JetBrains Mono

Testing

```
npm test
```

Tests cover: - MathJax prerendering output - Error handling in `prerender-math.js` and `render-pdf.js` - Scoped `\tag{}` replacement (code blocks untouched) - Script architecture enforcement (`@mathjax/src` usage, display-math block wrapper, Chrome flags) - End-to-end pipeline with PDF generation

Troubleshooting

“Pandoc not found” Ensure Pandoc is installed and available in your system PATH.

“MathJax package warnings” The script intentionally uses a minimal TeX package set. Warnings about `newcommand`, `ams`, `require`, and `autoload` can be ignored; the core rendering still works.

“PDF is blank / equations missing” - Ensure you have internet access for MathJax font loading (CDN) - Check `temp/*_step2.html` to verify MathJax rendered `mjx-container` elements - Use `-KeepTemp` to inspect intermediate files

“Images not showing in PDF” - Use Markdown image syntax `![alt](path)`, not link syntax - Ensure image paths are relative to `src/`

“Fonts look wrong offline” Google Fonts are embedded via `embed-resources: true` in `defaults.yaml`, but MathJax CHTML fonts still load from CDN. Offline builds may fall back to system fonts for math glyphs.

Configuration

Key settings are in `defaults.yaml` and `scripts/render-pdf.js`:

- Margins: `scripts/render-pdf.js:28-37`

- Math rendering: `scripts/prerender-math.js:54-65`
- Pandoc resource path: `defaults.yaml:12-13`